

DES QUESTIONS EN TP 2 DE OOP JAVA

Exercice 1 — Plus longue sous-séquence croissante

1. Pourquoi $O(n^2)$ est acceptable pour $n \leq 1000$?

Pour $n = 1000$, ça fait 1 million d'opérations. Un ordinateur fait ça en quelques millisecondes. Donc c'est assez rapide.

2. Existe-t-il plus rapide ?

Oui, $O(n \log n)$ avec une recherche binaire. Mais c'est plus compliqué à coder.

3. Si on remplace $<$ par $\leq$?

On obtient la version non-décroissante. Les doublons comptent : $\{3,3,3,3\}$ donne 4 au lieu de 1.

4. Comment adapter pour long, double, ou des objets ?

On change le type. Pour double, on utilise `Double.compare` (à cause des NaN). Pour des objets, on utilise `compareTo` ou un `Comparator`.

Exercice 2 — Tableau pivot

1. $O(n^2)$ vs $O(n)$?

Naïf : n^2 comparaisons (1 million pour $n=1000$). Efficace : $3n$ (≈ 3000). Donc l'efficace est ~ 300 fois plus rapide.

2. Pourquoi $\text{prefixMax}[i-1] \leq t[i]$?

$\text{prefixMax}[i-1]$ est le maximum de gauche. Si le max est $\leq t[i]$, alors tous les éléments de gauche le sont aussi.

3. Pourquoi $\text{suffixMin}[i+1] \geq t[i]$?

$\text{suffixMin}[i+1]$ est le minimum de droite. Si le min est $\geq t[i]$, alors tous les éléments de droite le sont aussi.

4. Si on remplace $\leq$ par $<$?

Les doublons cassent tout : $\{3,3,3,3\}$ ne donne plus aucun pivot.

5. Un seul passage ?

Non, car il faut voir la gauche (déjà vue) et la droite (pas encore vue). Mais on peut faire en 2 passages.

Exercice 5 — Permutation circulaire

1. Pourquoi "permutation" ne suffit pas ?

Parce que ça n'impose aucun ordre. 3 1 2 4 5 a tous les nombres 1..5 mais ce n'est pas une rotation. Il faut aussi vérifier l'ordre.

2. Pourquoi chercher la position de 1 ?

Parce que 1 est le début logique de la rotation. On sait d'où partir → $O(n)$ au lieu de tester n rotations ($O(n^2)$).

3. Pour 10, 20, ..., 10n ?

Même algorithme. On remplace 1 par 10 :

- On cherche 10 au lieu de 1
- On attend $(k+1) * 10$ au lieu de $k+1$

4. Détecter doublons et manquants en même temps ?

Oui, c'est possible mais plus compliqué. La version en 3 étapes (vérifier permutation, chercher 1, vérifier ordre) reste plus claire.

Exercice 6 — Kadane

1. Naïf vs Kadane ?

Naïf : on teste tous les segments ($\sim n^2$) → $O(n^3)$ ou $O(n^2)$.

Kadane : un seul passage. Un segment qui finit à i soit prolonge le précédent, soit repart de $t[i]$ → $O(n)$.

2. Quand réinitialiser sommeCourante ?

Quand elle est négative. Un passé négatif ne fait que réduire la somme → mieux vaut repartir de $t[i]$.

3. Tableau tout négatif ?

Le résultat est le plus grand (le moins négatif). Ex. $[-4, -2, -7] \rightarrow -2$.

On initialise à $t[0]$ (pas 0), sinon on retournerait 0 alors que le segment doit être non vide.

4. Longueur minimale ≥ 2 ?

On initialise avec $t[0]+t[1]$ et on redémarre avec $t[i-1]+t[i]$. Toujours $O(n)$.

Exercice 7 — Boyer-Moore

1. Deux majoritaires impossibles ?

Chacun devrait apparaître plus de $n/2$ fois → total $> n$. Impossible.

2. Boyer-Moore vs HashMap ?

HashMap : mémoire $O(k)$ (k = valeurs distinctes, pire cas $O(n)$).

Boyer-Moore : mémoire $O(1)$, juste 2 variables.

3. Pourquoi la 2^e passe ?

Le vote donne un candidat, pas une certitude. Sur [1,2,3,4], le candidat est 4 mais il n'est pas majoritaire. La 2^e passe vérifie.

4. Sans vérification ?

On retournerait des résultats faux quand il n'y a pas de majoritaire.

5. Pour un dominant à $n/3$?

On garde 2 candidats et 2 compteurs. À la fin, on vérifie les 2.

6. Un seul candidat suffit pour $n/3$?

Non. Deux éléments peuvent chacun dépasser $n/3$. En général, pour n/k , il faut $k-1$ candidats.

Exercice 8 — Nombres absents

1. Pourquoi le tableau de booléens garantit tout ?

On marque chaque valeur vue. À la fin : $vu[k] = \text{true} \rightarrow$ présent ; $vu[k] = \text{false} \rightarrow$ manquant.
On parcourt tous les $1..n \rightarrow$ aucun oubli.

2. Valeurs hors [1..n] ?

On les ignore (comme dans le code), ou on lève une exception, ou on les compte comme manquantes.

3. Intervalle [a..b] ?

Même principe : vu de taille $b-a+1$, on décale avec $vu[x-a]$, on parcourt k de a à b .

4. Solution en place ?

Avantage : $O(1)$ mémoire.

Risques : modifie le tableau, code plus complexe, moins lisible.

Exercice 9 — Diagonales

1. Pourquoi $O(n)$ et pas $O(n^2)$?

On ne parcourt que les $2n$ éléments des diagonales, pas les n^2 de la matrice.

2. Matrice non carrée ?

$m[i][n-1-i]$ n'a plus de sens. On peut lever une exception ou retourner -1.

3. Diagonale principale seule ?

Une méthode dédiée :

java

```
for (int i = 0; i < n; i++) somme += m[i][i];
```

4. Toutes les diagonales parallèles ?

Chaque diagonale a $i - j = d$ constant. On parcourt d de $-(n-1)$ à $n-1$. Complexité $O(n^2)$.

Exercice 10 — Carré magique

1. Pourquoi la 1ère ligne comme référence ?

Si le carré est magique, toutes les sommes sont égales. Donc n'importe quelle ligne fait l'affaire. On prend la 1ère par simplicité.

2. On vérifie 1..9 distincts ?

Non, seulement la somme égale. $\{\{4,4,4\},\{4,4,4\},\{4,4,4\}\}$ est considéré magique alors que ce n'est pas un vrai carré magique.

3. Comment imposer 1..9 ?

Ajouter `contient1a9(m)` avec un `boolean[10]` : vérifier $1 \leq v \leq 9$ et pas de doublon.

4. Généraliser à $n \times n$?

Remplacer les 3 par n et utiliser des boucles `for` pour lignes, colonnes, diagonales.

5. Complexité $n \times n$?

$O(n^2)$ en temps (on parcourt toutes les lignes et colonnes). $O(1)$ en mémoire.